

Håndterer ikke feil input, og displayer ikke prossessert element.

Fourth Assignment

29.10.2024

Sent **8/10 Points**

Forsøk 1 



Se gjennom tilbakemelding

30.10.2024

Forsøk 1 Poengsum:

8/10



Vis tilbakemelding

Anonym vurdering: **nei**

3 tillatte forsøk

22.10.2024 til 30.10.2024

▼ Detaljer

Assignment: Print Job Scheduling with Priority Queue Using Max-Heap

Assignment Description:

In this assignment, you will explore the concept of print job scheduling and implement a system that manages print jobs using a **priority queue** based on a **max-heap**. The objective is to create a program that allows users to add print jobs with different priorities and efficiently process them according to their priority.

Tasks:

Task 1: (5 points)

Design and implement a **priority queue** data structure using a **max-heap** to support print job scheduling. You should define the necessary attributes and methods for the priority queue and implement methods for inserting a print job into the queue and processing print jobs based on their priority.

- Higher-priority print jobs (jobs with a higher priority number) should be processed before lower-priority jobs.
- The system should efficiently manage the insertion and extraction of print jobs using the max-heap.
- Create a **print job structure** that includes the job's name and priority (an integer representing priority).
- Implement error handling for potential issues such as invalid input.

Task 2: (3 points)

Develop a mechanism for users to **add print jobs** to the queue along with their priority. The system should automatically manage the prioritization of jobs based on their priority values, meaning that jobs are inserted into the queue without the user needing to worry about order.

- Implement a user-friendly interface to accept print job names and priority levels from users.
- After each job insertion, display the next print job with the highest priority (without processing it yet).
- Ensure that each print job has a **unique name**.
- Display an error message if the user tries to insert a job with a **duplicate name**.

Task 3: (2 points)

Implement a feature that allows users to **update the priority** of an existing print job. The system should prompt the user to enter the name of the print job and its new priority, and then **reposition** the job in the queue based on its updated priority.

- The updated print job should be correctly repositioned in the max-heap.
- Display the updated order of the print jobs after the priority change.

Documentation and Submission

Your submission should include:

- The source code (with appropriate comments explaining the code logic).
- A brief report explaining:
 - The program's functionality.
 - Examples of how to use it.
 - Testing process and results.

Submit your C++ code files and the report as a single compressed file (ZIP file) through the Canvas platform.

Grading Criteria:

- Correct implementation of the Print Job Scheduling with Priority Queue Using Max-Heap (4 Points).
- Effective error handling (1 Point).
- Correct implementation of Task 2 (3 Points).
- Correct implementation of Task 3 (2 Points).

Example to Test the Program:

Scenario: You are managing a print queue with multiple print jobs, each having different priorities. Higher-priority jobs should be printed first.

Input Sequence:

- **Insert Print Job:**
 - Job Name: "Thesis"
 - Priority: 4
- **Insert Print Job:**
 - Job Name: "Project"
 - Priority: 5
- **Insert Print Job:**
 - Job Name: "Report"
 - Priority: 3
- **Insert Print Job:**
 - Job Name: "Assignment"
 - Priority: 2
- **Display Print Job with Highest Priority:** After all insertions
- **Process (Print) the Job with the Highest Priority**
- **Insert Print Job:**
 - Job Name: "Invoice"
 - Priority: 6
- **Update Print Job Priority:** Update "Thesis" priority to 7

Expected Output:

After step 1:

Queue contains: ["Thesis"]

Highest priority: **Thesis (Priority: 4)**

After step 2:

Queue contains: ["Project", "Thesis"]

Highest priority: **Project (Priority: 5)**

After step 3:

Queue contains: ["Project", "Thesis", "Report"]

Highest priority: **Project (Priority: 5)**

After step 4:

Queue contains: ["Project", "Thesis", "Report", "Assignment"]

Highest priority: **Project (Priority: 5)**

Step 5: Display: **Project (Priority: 5)** is the next print job to be processed.

Step 6: "Project" is printed and removed from the queue.

Queue contains: ["Thesis", "Report", "Assignment"]

Highest priority: **Thesis (Priority: 4)**

After step 7:

Queue contains: ["Invoice", "Thesis", "Report", "Assignment"]

Highest priority: **Invoice (Priority: 6)**

Step 8: After updating "Thesis" priority to 7, the queue is:

Queue contains: ["Thesis", "Invoice", "Report", "Assignment"]

Highest priority: **Thesis (Priority: 7)**
